Two routes to interpretability

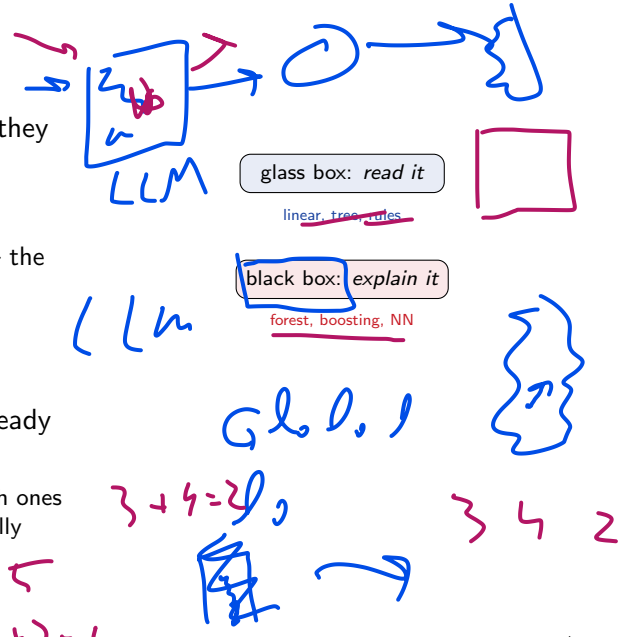
You built models that work — but *why* do they predict what they do?

Two routes:

- **Use a glass-box model you can read** — the model *is* the explanation. **(this deck)**
- **Explain a black box afterwards** with model-agnostic tools. (decks 2–3)

Today: the two transparent families you already know — **linear models** and **trees**.

Why bother? To **trust** a model, **debug** it, and catch ones that *cheat* — like deck 3's "wolf" detector that really keys on *snow*.



Predict-first: which one can you actually read?

A linear model and a random forest reach the **same accuracy** on the bike-rental data. Which can you *read* — and how?

Predict-first: which one can you actually read?

A linear model and a random forest reach the **same accuracy** on the bike-rental data. Which can you *read* — and how?

- **Linear model:** yes — each feature has one coefficient.
- **A single tree:** yes — follow the splits from root to leaf (a rule).
- **A random forest: no** — hundreds of trees, no readable whole. But it still hands you a built-in *importance* number (which, we'll see, is biased).

Running example: **bike sharing** — predict daily rental count from weather & calendar features.

Outline

Reading a linear model

Reading trees

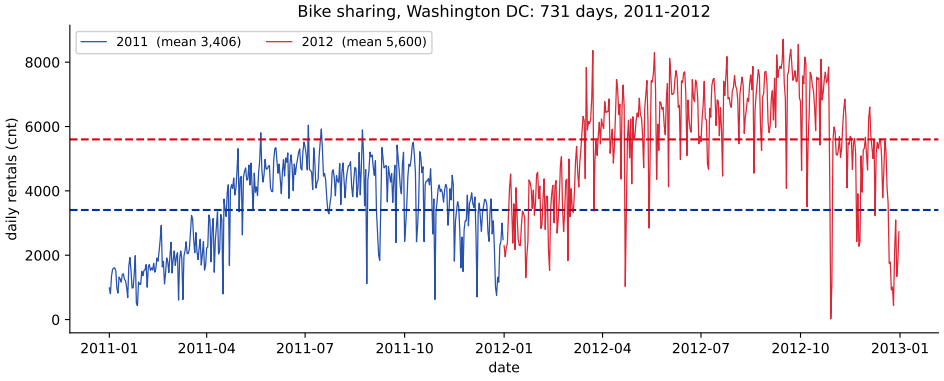
Rule-based models

So which features matter?

The running example: bike sharing

The task: predict how many bikes are rented in Washington DC on a given day.

UCI Bike Sharing — 731 daily records, 2011-01-01 to 2012-12-31. Target `cnt`: total rentals that day, from 22 to 8714 (mean $\approx$ 4500).



Two patterns to keep in mind: rentals grew +64% from 2011 to 2012 (the service was expanding), and they swing hard with the **seasons**. That growth is why `yr` tops almost every importance ranking in this chapter.

The 11 features — and what `atemp` is

`season`	1 winter, 2 spring, 3 summer, 4 fall
`yr`	**0 = 2011, 1 = 2012**
`mnth`	month, 1–12
`holiday`	is it a holiday?
`weekday`	day of week, 0–6
`workingday`	neither weekend nor holiday
`weathersit`	1 clear, 2 mist, 3 light rain/snow
`temp`	air temperature
`atemp`	**"feels-like" temperature**
`hum`	humidity
`windspeed`	wind speed
`cnt`	**the target** — rentals that day

`atemp` **is the apparent temperature** — what the air *feels* like once wind and humidity are accounted for. It is the same weather, measured a second way.

On this data $\mathrm{corr}(\texttt{temp}, \texttt{atemp}) = \mathbf{0.99}$. **Remember that number** — it causes trouble in all three decks of this chapter.

All four weather columns are rescaled to $[0, 1]$, so the numbers are not degrees. For temperature T in Celsius, $\texttt{temp} = (T + 8)/47$.

So when a tree splits at $\texttt{temp} \leq 0.43$, it means **"colder than about** 12°**C"**.

Reading a linear model

The most transparent model: one number per feature.

Coefficients as importance

A linear model $\hat{y} = \theta_0 + \sum_j \theta_j x_j$ tells you everything directly:

- **Sign** of θ_j = direction; **magnitude** = effect per $+1$ unit of x_j.
- **Standardize first** to compare — a raw coef of 5000 on `temp` (range 0–1) vs 2 on `hum` is *not* "2500× more important"; that's units (L12). Standardized, $|\theta_j|$ is fair: on bike `yr` & `atemp` top it.
- Classification analog: e^{θ_j} = **odds ratio** (L11).

No extra method needed: a (standardized) coefficient *is* a built-in importance and effect, in one number.

Two words for the whole chapter: main effect, interaction

Main effect of a feature: what it does to the prediction **on its own**, averaging over everything else.

A linear model is *nothing but* main effects — each θ_j **is** one.

Interaction: when one feature's effect **depends on the value of another**.

Then no single coefficient can tell the story — the answer to "what does `temp` do?" becomes *"it depends"*.

Made concrete: suppose warm weather lifted rentals a lot on `workingdays` but barely at all on holidays.

- `temp`'s effect is no longer *one* number — it is one number per kind of day.
- That is an **interaction** between `temp` and `workingday`.

$$\hat{y} = \theta_0 + \theta_1 x_{\text{temp}} + \theta_2 x_{\text{work}} + \underbrace{\theta_3 x_{\text{temp}} x_{\text{work}}}_{\text{the interaction term}}$$

A linear model has interactions **only if you write them in by hand**.

Deck 2 makes this precise (the *functional ANOVA* decomposition) and puts a **number** on how much interaction a model actually has (Friedman's H).

Linear models: the caveats

Only additive, linear effects

- One straight-line effect per feature; no interactions or curves unless you engineer them.

Correlated features split the credit

- `temp` and `atemp` (feels-like) are nearly the same column, so the model has to divide *one* effect between *two* coefficients — next frame.

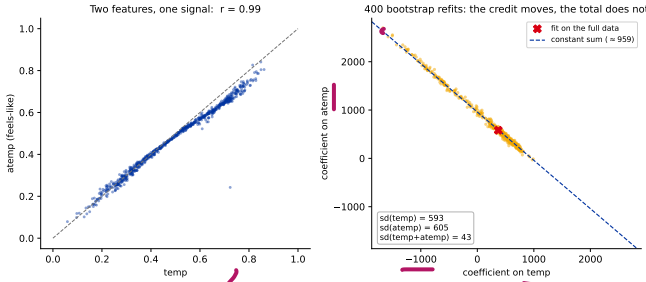
A coefficient answers "how does the prediction move with this feature, *holding the others fixed*" — which is exactly what breaks when features move *together*.

Correlated features split the credit

`temp` and `atemp` carry **one signal** ($r = 0.99$). The fit must divide that effect between them, and *where* it lands is close to arbitrary.

Across 400 bootstrap refits each coefficient swings by $\approx \pm 600$, and `temp` comes out **negative on 32%** of them.

Their **sum** hardly moves: $\mathrm{sd} = 43$, about $14\times$ steadier than either part alone.



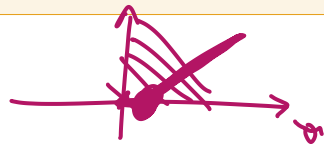
The tell: drop `atemp` and `temp` jumps to $+$**943** — almost exactly the pair's sum ($+953$). **A small coefficient is not a small effect.**

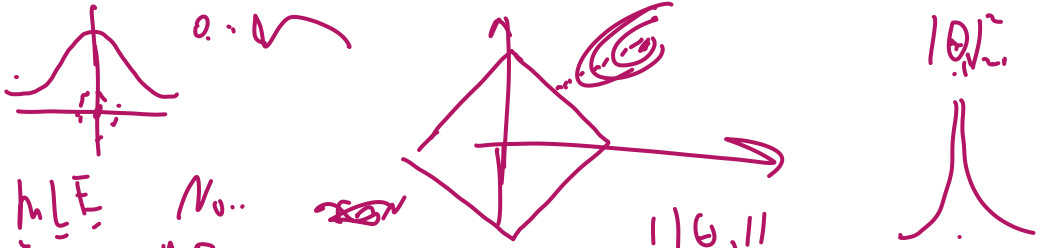
Lasso: the model selects its own features

L1 (Lasso) regularization shrinks coefficients toward zero — and drives many to **exactly zero** (callback: Ch. 2 regularization).

- Non-zero coefficients = the features the model *kept* — a built-in, sparsity-based **feature selection**.
- Stronger penalty ⇒ fewer survivors. **On bike (scaled):** the CV-tuned Lasso keeps 10/11; turn the penalty up and survivors drop $10 \to 4 \to 1$ — **sparsity is a knob you set**.

Caveat: among correlated features, Lasso keeps *one* and zeros the rest *arbitrarily* — "not selected" does not mean "unimportant."





Reading trees

One tree you can read; a forest you cannot — but it still scores features.

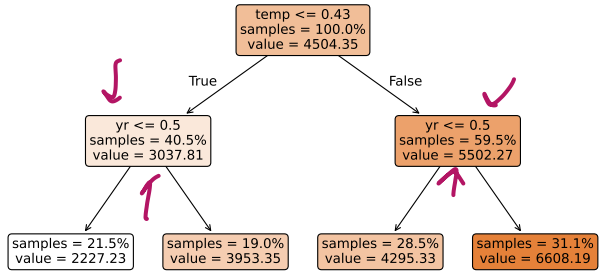
A single tree is a glass box

Each **root-to-leaf path is a rule**:

warm day ($\texttt{temp} > 0.43$*) in year 2 (*$\texttt{yr} = 1$*; 0=2011, 1=2012)* $\rightarrow$ *predict* ~ 6608 *rentals.*

- Fully transparent — if it stays shallow.
- Depth $\uparrow \Rightarrow$ accuracy $\uparrow$ but readability $\downarrow$.

A shallow regression tree on bike rentals (read a path as a rule)



How a split is chosen: reduce impurity

Recall from the trees lecture (ch. 4): a tree picks the split that most reduces **impurity** = parent − *weighted* child. Same idea, two rulers — one worked number each:

Classification: information gain (entropy H)

Toy: parent 5/5 $\Rightarrow H = 1.0$; a 4/1 child has $H = 0.72$.

$$\text{gain} = 1.0 - \underbrace{0.72}_{\text{weighted child}} = \mathbf{0.28}$$

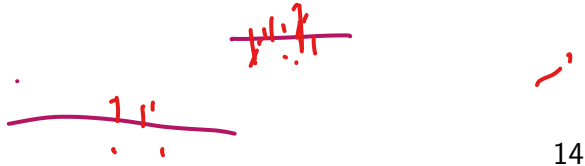
The split with the biggest reduction wins.

Regression: variance reduction (bike root)

Split `temp`≤ 0.43 ($n = 731$):

$$\text{reduction} = \underbrace{3.75\text{M}}_{\text{parent}} - \underbrace{2.32\text{M}}_{\text{weighted child}} \approx \mathbf{1.43}\text{M}$$

($\approx 38\%$ of the parent variance)



Forests and boosting: unreadable, but they score features

A random forest is **hundreds of trees** fitted in parallel; a **boosted** model (GBM, XGBoost, LightGBM, CatBoost — ch. 4) is hundreds fitted *in sequence*. Neither has a readable whole. **Both** still hand you a built-in **feature importance**:

Sum each feature's impurity reduction (info gain / variance reduction) over *all* its splits, summed or averaged across the trees ⇒ one importance per feature.

And in both, it is biased the same three ways:

- Favors **high-cardinality / continuous** features (more places to split).
- **Dilutes** across correlated features (credit shared between `temp`, `atemp` — the same splitting we just saw for coefficients).
- Computed on **training** data ⇒ can reward overfitting.

Careful: your library picks the definition for you

"Feature importance" is not one quantity. `feature_importances_` means something **different** depending on what you fitted:

Model	**What `feature_importances_` returns**
sklearn `RandomForest`, `GradientBoosting`	impurity reduction (Gini / variance)
sklearn `HistGradientBoosting`	nothing — the attribute does not exist
XGBoost	**gain** (avg. improvement per split)
LightGBM	**split count** (how often it was used)

On a toy where *only* x_1 carries signal: XGBoost's gain gives x_1 **85%** of the total, while LightGBM's split count gives it 121 splits against 46/39/14 for three pure-noise features — **45%** of the "importance" handed to noise. Counting splits *is* the cardinality bias.

sklearn dropped the attribute from `HistGradientBoosting` on purpose and points you at **permutation importance** instead — which is exactly where deck 2 starts.

Reading these in sklearn

```
# linear: standardized coefficients as importance
Xs = StandardScaler().fit_transform(X)
coef = LinearRegression().fit(Xs, y).coef_          # |coef| =
    importance
sel  = make_pipeline(StandardScaler(), LassoCV()) # SCALE, then L1 (
    scale-sensitive)
kept = sel.fit(X, y)[-1].coef_ != 0                 # the features
    Lasso keeps

# trees: built-in (impurity) importance + read one tree
RandomForestRegressor().fit(X, y).feature_importances_
plot_tree(DecisionTreeRegressor(max_depth=3).fit(X, y))

# boosting: SAME attribute name, different definition per library
XGBRegressor().fit(X, y).feature_importances_       # gain
LGBMRegressor().fit(X, y).feature_importances_      # split COUNT -
    different!
```

Rule-based models

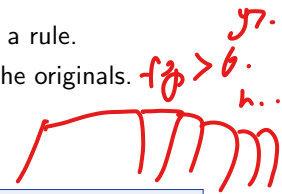
Rules you can read — a tree's logic with a linear model's sparsity.

RuleFit: turn rules into features

A single tree is readable but brittle; a forest is accurate but unreadable. **RuleFit** (Friedman & Popescu, 2008) takes the middle road: **keep the trees' logic, throw away their structure**.

The whole method is three steps, one per frame from here:

1. **Mine rules** from a tree ensemble — every root-to-node path becomes a rule.
2. **Add them as columns** — each rule is a new binary feature, next to the originals.
3. **Lasso** the lot — the penalty decides how many terms survive.

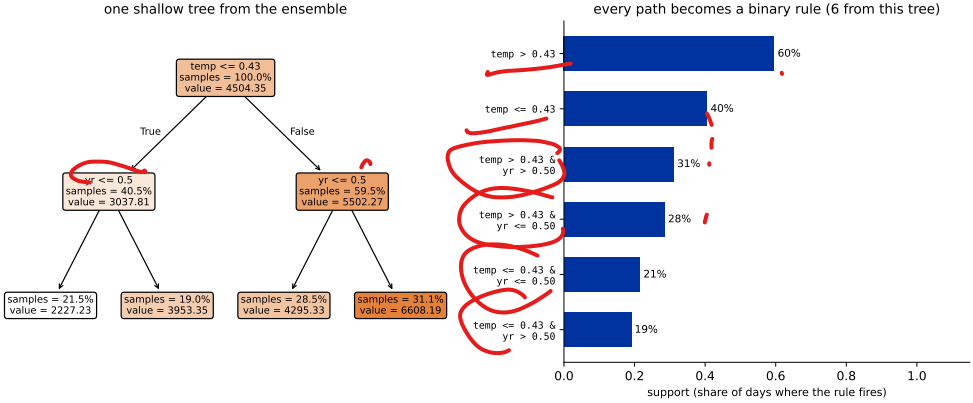


RuleFit = **rules from trees** + **Lasso selection** — the two glass boxes from this deck, combined into one readable model.

Step 1: mine rules from the ensemble

Walk a tree from the root. Every node you pass is a **conjunction of split conditions** — make it a binary feature: 1 if the row satisfies all of them, 0 otherwise.

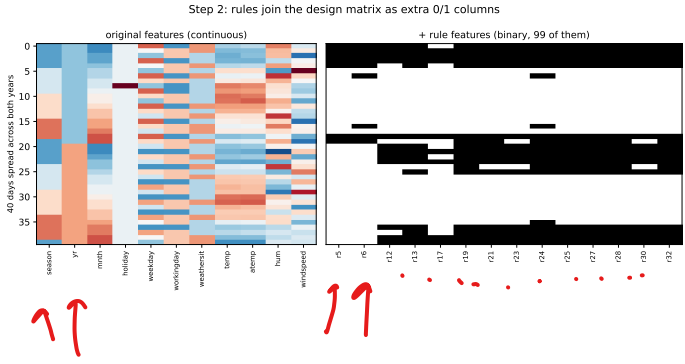
Our depth-2 tree from earlier yields **6** rules: two with one condition, four with two. **Support** = the share of days on which the rule fires. A real run repeats this over the whole ensemble — on bike that is **99 candidate rules**.



Step 2: every rule becomes a 0/1 column

The rules are simply **appended** to the design matrix. Bike goes from 11 columns to $11 + 99 = \mathbf{110}$.

- Nothing about the fitting is special — rules are **just features** now.
- The **original columns stay**: that is what lets RuleFit express a smooth trend without stacking dozens of step-shaped rules to approximate it.

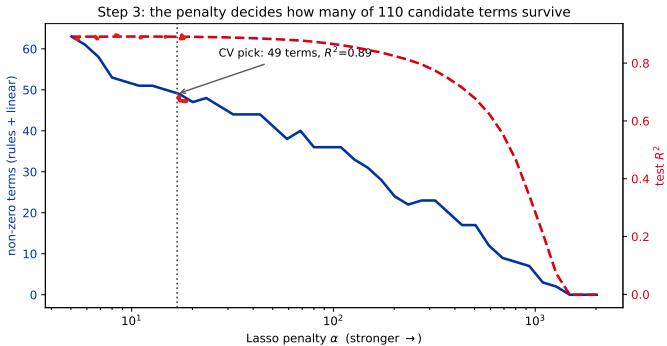


Step 3: Lasso decides how many survive

Fit a Lasso on all 110 terms. Because most rules are redundant, the penalty α acts as a **readability dial**:

- CV's pick: 49 terms, test $R^2 = 0.89$ — it has caught the forest (0.88), and is far too long to read.
- Turn α up: 24 terms $\rightarrow$ 0.82; 9 terms $\rightarrow$ 0.56.

You choose where to sit on this curve. RuleFit does not choose for you — and CV will always pull toward the accurate, unreadable end.

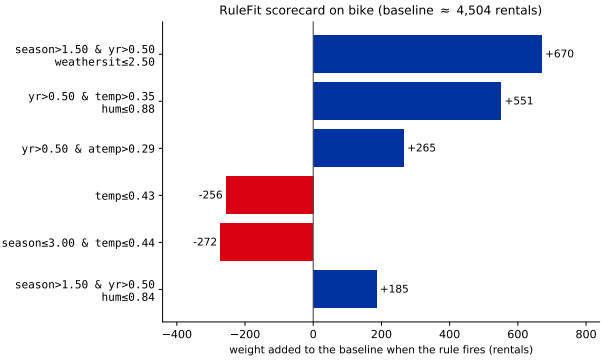


Reading a RuleFit model: a scorecard

Read it like a **scorecard**: start at the baseline ($\sim$4500 rentals, the average), then **add the weight of every rule that fires** for this day.

A warm, fair 2012 day fires the big **positive** rules (+670, +551, ...) and stacks well above average; a cold day ($\texttt{temp} \leq 0.43$) fires the **negative** ones. This 13-rule fit scores $R^2 \approx 0.65$ — a deliberately sparse point on the last frame's curve, traded for a model that fits on one slide.

Watch out: more rules $\Rightarrow$ more accurate but less readable (a denser RuleFit catches the forest's ~ 0.88); overlapping rules **share credit**, so one weight is not a feature's full effect.



So which features matter?

Three glass boxes, three answers — and the disagreement is worth reading.

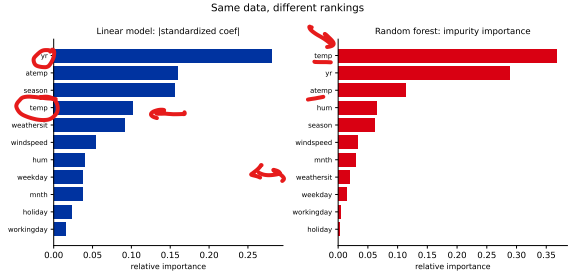
Do the two methods agree?

Same bike data, same goal — but the linear model and the random forest do *not* pick the same top features.

They don't:

- Linear: `yr`, `atemp`, `season` on top.
- Forest: `temp`, `yr`, `atemp` on top.

Part of the gap is the **impurity bias** we just saw; the rest is that they genuinely **measure different things** (a linear effect vs. impurity reduction), splitting the `temp`/`atemp` correlation differently.

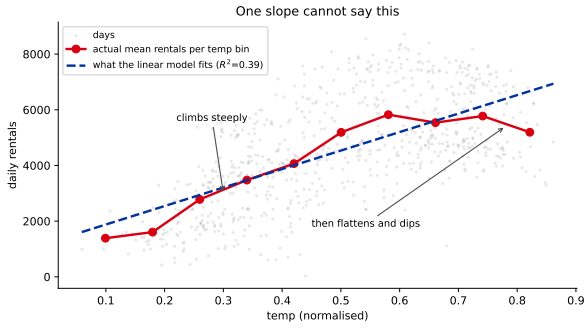


Importance is method-dependent — always say *which* importance you mean.

The disagreement is information, not noise

Don't just pick a winner. **Which way** they disagree tells you about the **shape** of the effect:

- **Trees high, linear low** ⇒ the effect is **curved or interaction-driven**. A single slope cannot express it, so the coefficient stays small.
- **Linear high, trees low** ⇒ a **smooth monotone** trend (trees need many splits to trace one), or credit **diluted** across correlated columns.



Bike `temp` climbs steeply, then **flattens and dips**. One line has to average the two, so the linear model under-rates it — the forest, which can bend, does not.

Use the gap as a **diagnostic**: it tells you which features deserve a **feature-effect plot** (deck 2) — which shows you the curve instead of making you infer it.

Recap

- **Linear models** are read directly: (standardized) **coefficients** = effect + importance; e^{θ_j} = odds ratio; **Lasso** selects features by zeroing them.
- **Correlated features share one effect** between several coefficients — unstable, sometimes sign-flipped. A **small coefficient is not a small effect**.
- **A single tree** is a glass box (paths = rules); splits are chosen by **impurity reduction** (information gain / variance reduction).
- **RuleFit** (Friedman & Popescu, 2008) mines rules from an ensemble, adds them as binary columns, then Lasso-selects a short weighted list you read as a **scorecard** — accuracy vs readability is a dial you set.
- **Forests and boosted models** are black boxes but emit a built-in **impurity importance** — fast, **biased** (cardinality, correlation, train-based), and **defined differently by each library** (gain vs split count).
- Different methods give **different rankings** — read the **disagreement** as a clue to the effect's shape. Importance is method-dependent, and *none of it is causation*.

Next: model-agnostic interpretability — **permutation importance** (fixes the cardinality & train-based biases — though correlation still bites) and **feature-effect curves**, which show the *shape* of a feature's influence, for *any* model.